UNIP - UNIVERSIDADE PAULISTA

TC1 - Introdução à programação estruturada

Ciência da Computação

UNIP

segundo semestre, 2026



UNIP - UNIVERSIDADE PAULISTA

TC1 - Introdução à programação estruturada

Ciência da Computação

UNIP

segundo semestre, 2026

Objetivo do modelo de documento

- Padronizar a organização do TC (clareza, completude e rigor).
- Garantir coerência entre:
  problema $\rightarrow$ **método** $\rightarrow$ **implementação** $\rightarrow$ **avaliação**.
- Facilitar avaliação (banca) e reprodutibilidade (outros conseguem repetir e verificar).
- Aderir às normas acadêmicas (ex.: ABNT, referências, formatação).

Um TC ideal não é só "o sistema funcionando", é:
evidência + método + análise + documento acadêmico

Elementos pré-textuais (ABNT) — visão geral

- Capa
- Folha de rosto
- Folha de aprovação
- Resumo (200–400 palavras)
- Palavras-chave (3 a 5)
- Abstract
- Keywords
- Lista de figuras
- Lista de tabelas
- Lista de siglas
- Sumário

Dica: o resumo deve explicitar problema, método, resultados e conclusão em poucas linhas.

Introdução — o que deve conter

- Contextualização do tema e cenário de aplicação.
- Motivação e relevância (técnica e/ou social).
- **Problema de pesquisa** (idealmente como pergunta).
- Objetivo geral e objetivos específicos.
- Justificativa (por que essa abordagem faz sentido).
- Organização do documento (o que vem em cada capítulo).

Exemplo de problema (formato pergunta):
Como otimizar o rastreamento de resíduos recicláveis utilizando blockchain para garantir transparência na cadeia de reciclagem?

Fundamentação teórica — papel no TC

- Demonstrar domínio conceitual: principais termos, modelos e técnicas.
- Sustentar escolhas do projeto (por que este algoritmo? por que esta arquitetura?).
- Cobrir:
  - Conceitos centrais do tema
  - Tecnologias envolvidas
  - Trabalhos relacionados
  - Comparação entre abordagens (quando pertinente)

Preferir fontes acadêmicas, artigos e documentação oficial; evitar depender só de blogs.

Exemplos de conteúdos possíveis (dependendo do tema)

- Algoritmos de busca (BFS, DFS, A*, heurísticas)
- IA e aprendizado (CNN, MLP, validação)
- Blockchain e rastreabilidade
- Engenharia de requisitos
- Arquitetura de software
- Testes e verificação
- Qualidade de software (ISO/IEC 25010)
- Métricas e avaliação

Metodologia — o que o leitor precisa conseguir repetir

- Tipo de pesquisa: aplicada, experimental, exploratória, etc.
- Processo de desenvolvimento (ex.: iterativo, Scrum, cascata).
- Ferramentas, linguagens, frameworks e ambiente de execução.
- Dados: fontes, bases, critérios de seleção, pré-processamento.
- **Critérios de avaliação** e **métricas** (antes dos resultados).

A metodologia define como você vai provar que sua solução funciona.

Metodologia — exemplos de métricas (por tipo de TG)

- **Busca/otimização:** tempo de execução, nós expandidos, qualidade da solução.
- **IA/classificação:** acurácia, precisão, recall, F1, matriz de confusão.
- **Sistemas:** latência, throughput, consumo de recursos, disponibilidade.
- **Qualidade:** mapeamento para ISO/IEC 25010 e métricas derivadas (quando aplicável).

Modelagem do sistema / solução

- Descrever a solução antes de implementar.
- Diagramas e modelos conforme o foco:
  - UML (casos de uso, classes, sequência)
  - Modelo entidade-relacionamento (banco de dados)
  - Arquitetura (camadas, componentes, fluxos)
  - Modelagem matemática (quando aplicável)

Modelagem não é “enfeite”: ela justifica decisões e facilita manutenção.

Desenvolvimento / Implementação — o que detalhar

- Estrutura do sistema (módulos, pacotes, camadas).
- Principais decisões técnicas e trade-offs.
- Algoritmos e heurísticas implementados.
- Banco de dados e APIs (se houver).
- Trechos de código **apenas quando relevantes** (o restante vai para apêndice).

Evite "print do código inteiro" no corpo do texto. Foque no que é decisivo.

Resultados — o que apresentar

- Evidências quantitativas (tabelas, gráficos, testes).
- Comparações: baseline, versões, trabalhos relacionados (quando possível).
- Repetibilidade: parâmetros, sementes, configurações, hardware.

Exemplo de tabela comparativa (busca):

Algoritmo	Tempo médio	Observação
BFS	0.45 s	Expande muitos nós
A*	0.12 s	Depende de heurística

Discussão — o que interpretar

- O que os resultados significam em relação ao objetivo?
- Limitações e ameaças à validade (dados, ambiente, viés, generalização).
- Impactos: técnicos, sociais e/ou de sustentabilidade (quando pertinente).
- O que faria diferente numa continuação do projeto.

Resultados sem discussão viram “relatório de execução”, não pesquisa aplicada.

Conclusão — estrutura recomendada

- Retomar problema e objetivos.
- Sintetizar a solução proposta.
- Principais resultados (com números, quando existirem).
- Contribuições do trabalho.
- Trabalhos futuros (próximos passos claros e viáveis).

Referências, apêndices e anexos

- **Referências:** padronização (ABNT NBR 6023 ou padrão exigido pelo curso).
- **Apêndices (opcional):** materiais produzidos pelo autor (código, instrumentos, detalhes técnicos).
- **Anexos (opcional):** materiais de terceiros (documentos, manuais, etc.).

Regra prática: não “esconder” informação essencial nos apêndices.

Critérios de qualidade (o que caracteriza um TC forte)

- Coerência: problema → método → implementação → avaliação.
- Rigor: métricas, experimentos, comparações, controle de variáveis.
- Clareza: escrita objetiva, figuras legíveis, capítulos bem amarrados.
- Originalidade: contribuição explícita.
- Reprodutibilidade: parâmetros, dados, versão de ferramentas, ambiente.
- Conformidade acadêmica: formatação e referências.

Checklist: 6 perguntas que o TG deve responder

1. Qual problema estou resolvendo?
2. Por que ele é relevante?
3. Qual solução proponho?
4. Como implementei?
5. Como validei?
6. Quais evidências demonstram que funciona?

Se uma dessas respostas estiver fraca, a banca vai sentir.